Stack Tecnológico y Métricas de Software del Sistema de Información y Control del Transmetro

Luis Orozco

Carné 3590 22 5339

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Aseguramiento de la Calidad del Software

23 de octubre de 2026

2

Stack Tecnológico y Métricas de Software

El Sistema de Información y Control del Transmetro (SIC Transmetro) centraliza la información de las líneas, estaciones y unidades del sistema de transporte de la Ciudad de Guatemala y municipios aledaños. El presente documento desarrolla dos aspectos del proyecto: el stack tecnológico, entendido como el conjunto de lenguajes, bibliotecas y servicios que el sistema emplea en cada capa, y el plan de métricas de software, que define qué se mide, cómo se mide y con qué propósito.

Esta es la segunda medición del proyecto. La primera se realizó el 2 de octubre de 2026, al cierre del Sprint 0. Entre ambas fechas se construyeron cuatro módulos adicionales con interfaz —monitoreo de capacidad y servicio, personal, mi expediente y mi cobertura— y se reconstruyó el tablero con mapa de la red e indicadores gráficos. Todos los valores que se presentan corresponden a la medición del 19 de octubre de 2026 y, cuando resulta pertinente, se contrastan con los de la medición anterior.

Primera Parte: Stack Tecnológico

El sistema adopta una arquitectura en tres capas desacopladas. La característica distintiva de su stack es el uso de un solo lenguaje de programación de aplicación, JavaScript, tanto en el navegador como en el servidor, con SQL y PL/pgSQL reservados para la capa de datos. Todos los componentes son de código abierto y carecen de costo de licencia.

La capa de presentación se comunica con la capa de aplicación mediante HTTPS en el puerto 443 con formato JSON, y la capa de aplicación se comunica con la capa de datos mediante el protocolo de PostgreSQL en el puerto 5432 con cifrado TLS.

Conviene señalar un resultado de la medición antes de presentar la tabla: la construcción de los cuatro módulos nuevos, del mapa de la red y de las tres gráficas no incorporó ninguna dependencia adicional. El stack es exactamente el mismo que el del Sprint 0, con las mismas versiones. El mapa y las gráficas se dibujan en SVG escrito a mano, decisión tomada porque una biblioteca de gráficas pesaría más que todo el código de la interfaz y porque un mapa que depende de un servicio externo deja de ser útil precisamente en la estación con conectividad intermitente.

Tabla 1

Stack tecnológico completo del sistema por capa

Capa	Componente	Versión	Función en el sistema
Presentación	React y React DOM	18.3.1	Construcción de la interfaz mediante componentes reutilizables
Presentación	React Router DOM	6.28.0	Enrutamiento de las quince pantallas y protección de rutas por módulo y nivel
Presentación	Vite	6.0.7	Servidor de desarrollo con recarga inmediata y empaquetado para producción
Presentación	Plugin de React para Vite	4.3.4	Transformación de la sintaxis JSX a JavaScript
Presentación	JavaScript con JSX	ECMAScript 2022	Lenguaje de la interfaz
Presentación	CSS3 propio	Estándar del W3C	Hoja de estilos de 556 líneas, adaptable desde 820 píxeles
Presentación	SVG propio	Estándar del W3C	Mapa de la red y tres gráficas, sin biblioteca de terceros
Aplicación	Node.js	20 LTS o superior	Entorno de ejecución de JavaScript en el servidor
Aplicación	Express	4.21.2	Marco de trabajo de la interfaz REST y encadenamiento de middlewares
Aplicación	JavaScript con módulos ES	ECMAScript 2022	Lenguaje del servidor
Aplicación	pg	8.13.1	Controlador de PostgreSQL con agrupación de conexiones
Aplicación	zod	3.24.1	Validación y conversión de tipos de los datos recibidos
Aplicación	jsonwebtoken	9.0.2	Emisión y verificación de los tokens de acceso y de refresco
Aplicación	bcryptjs	3.0.2	Cifrado de contraseñas con factor de coste 12
Aplicación	Helmet	8.0.0	Cabeceras de seguridad HTTP
Aplicación	CORS	2.8.5	Control de orígenes permitidos
Aplicación	express-rate-limit	7.5.0	Límite de peticiones por dirección, con restricción adicional en el inicio de sesión

Aplicación	morgan	1.10.0	Registro de peticiones
Aplicación	dotenv	16.4.5	Carga de variables de entorno
Datos	PostgreSQL	14 o superior, verificado en 18.3	Motor relacional; almacena los datos y garantiza las reglas de negocio
Datos	SQL	SQL:2016, dialecto PostgreSQL	Definición del esquema, consultas y vistas
Datos	PL/pgSQL	Nativo del motor	15 funciones y 29 disparadores de reglas de negocio y auditoría
Datos	Extensión btree_gist	Del motor	Habilita las restricciones de exclusión que combinan igualdad con rangos de tiempo (RF-04 y RF-06)
Datos	Extensión citext	Del motor	Comparación de usuario y correo sin distinguir mayúsculas
Datos	Extensión pgcrypto	Del motor	Funciones criptográficas; el cálculo de hash se realiza actualmente en la capa de aplicación
Calidad	Vitest	2.1.9	Ejecución de pruebas unitarias y de integración
Calidad	Supertest	7.0.0	Pruebas de los puntos de acceso REST sin levantar el servidor
Calidad	PGlite	0.5.8	Ejecución de PostgreSQL dentro de las pruebas y como servidor local, sin instalar un motor
Calidad	ESLint	9.17.0	Análisis estático del código
Calidad	Mermaid CLI	11	Generación de los diagramas a partir de fuentes de texto versionadas
Calidad	Git	2.51	Control de versiones del código, del guion y de los diagramas
Calidad	npm	10 o superior	Gestión de dependencias y ejecución de guiones
Despliegue	Hosting estático o red de distribución de contenido	Servicio en la nube	Entrega de los archivos generados por Vite
Despliegue	Balanceador de carga	Servicio con certificado TLS	Terminación de HTTPS y distribución entre instancias
Despliegue	Servidor de aplicación	Node.js 20 LTS	Ejecución de la interfaz REST; sin estado en memoria, admite varias instancias

Despliegue	Base de datos administrada	PostgreSQL	Respaldo diario, registro continuo de transacciones y réplica

Nota. Las capas de presentación, aplicación y datos constituyen el producto desplegado. Las herramientas de calidad determinan cómo se construye y se verifica el sistema, pero no se despliegan. El esquema contiene 33 tablas, 12 vistas, 77 llaves foráneas y 80 restricciones de integridad. No se emplea biblioteca de componentes, framework de CSS ni biblioteca de gráficas de terceros.

Tabla 2

Requisitos del entorno de ejecución

Requisito	**Versión mínima**	**Ámbito**
Node.js	20	Desarrollo y servidor de producción
npm	10	Desarrollo
PostgreSQL	14	Base de datos, local o administrada
Navegador web	Versión vigente de Chrome, Edge, Firefox o Safari	Equipos de estación y dispositivos móviles
Git	2.40	Desarrollo

Nota. Los equipos de estación no requieren instalación de software adicional, puesto que la aplicación se ejecuta en el navegador. La ausencia de estado en memoria en el servidor de aplicación es la condición que permite escalar de forma horizontal.

Segunda Parte: Métricas de Software

Una métrica de software es una medida cuantitativa de un atributo de un producto, de un proceso o de un proyecto de software. Su propósito no es describir, sino permitir decisiones: establecer si un entregable cumple los criterios de aceptación, detectar desviaciones de forma temprana y sustentar las estimaciones de esfuerzo (Pressman y Maxim, 2020).

Este proyecto clasifica sus métricas en tres grupos, conforme a lo que cada una mide.

Tabla 3

Clasificación de las métricas del proyecto

Grupo	**Qué mide**	**Ejemplo en este proyecto**
Métricas de producto	Atributos del software construido	Tamaño del código, cantidad de restricciones en la base de datos, tiempo de respuesta
Métricas de proceso	Atributos de la forma en que se construye	Defectos detectados por fase, cumplimiento de la definición de terminado
Métricas de proyecto	Atributos del avance y del esfuerzo	Requerimientos implementados por sprint, velocidad del equipo

Nota. Elaboración propia con base en Pressman y Maxim (2020).

Las características de calidad evaluadas se alinean con el modelo de calidad del producto de la norma ISO/IEC 25010 (ISO/IEC, 2011), en particular las de adecuación funcional, eficiencia de desempeño, fiabilidad, seguridad y mantenibilidad.

Métricas de Producto: Tamaño

Las mediciones se realizaron el 19 de octubre de 2026 sobre el repositorio, excluyendo dependencias externas y archivos generados. El conteo de líneas incluye comentarios y líneas en blanco.

Tabla 4

Tamaño del código fuente por lenguaje

Lenguaje	**Archivos**	**Líneas**	**Participación**
JavaScript (servidor y utilidades)	47	4,737	37 %
JavaScript con JSX (interfaz)	24	3,494	27 %
SQL y PL/pgSQL	12	2,423	19 %
Mermaid (diagramas)	24	1,641	13 %
CSS	2	557	4 %
HTML y JSON	4	89	menos de 1 %
Total	113	12,941	100 %

Nota. Total de 12,941 líneas en 113 archivos, frente a 10,040 líneas en 96 archivos en la medición del 2 de octubre de 2026: un incremento del 29 %. Los diagramas en Mermaid se contabilizan como código porque se versionan y se revisan igual que el resto del repositorio. La documentación en formato Markdown suma 1,136 líneas adicionales, no incluidas en el total.

Tabla 5

Tamaño del modelo de datos y de la interfaz de programación

Elemento	**2 de octubre**	**19 de octubre**
Tablas	32	33
Vistas	12	12
Tipos enumerados	31	31
Funciones en PL/pgSQL	15	15
Disparadores	29	29
Índices creados explícitamente	37	37
Llaves foráneas	76	77
Restricciones CHECK	47	49
Restricciones UNIQUE	28	28
Restricciones EXCLUDE	3	3
Columnas generadas	4	4
Módulos del servidor	9	12
Puntos de acceso REST	51	75
Pantallas de la aplicación web	11	15

Nota. Valores obtenidos por consulta al catálogo del sistema tras ejecutar el guion completo, y por conteo sobre el código fuente. La tabla nueva es parametro_sistema, que externaliza los umbrales del negocio —espera mínima, ocupación mínima, umbral de sobrecupo, tarifas y ventana de transbordo— de modo que cambiarlos no exija modificar el código. Las dos restricciones CHECK nuevas y la llave foránea nueva pertenecen a esa tabla y a los campos de tarifa, horario y frecuencia agregados a la tabla de líneas. La medición anterior reportó cuatro restricciones EXCLUDE; la verificación contra el catálogo del motor confirma que son tres, y el valor se corrige en esta edición.

La relación entre restricciones declarativas y tamaño del esquema constituye en sí misma un indicador: 80 restricciones de integridad para 33 tablas, es decir, 2.4 restricciones por tabla. Cuanto mayor es esa proporción, menor es la cantidad de validaciones que deben programarse y mantenerse en la capa de aplicación.

Métricas de Producto: Calidad

Tabla 6

Fichas de las métricas de producto

Métrica	Fórmula	Unidad	Meta	Instrumento	Frecuencia
Cobertura de pruebas en servicios	Líneas ejecutadas por las pruebas entre líneas totales de los módulos, por cien	Porcentaje	Igual o mayor a 70 %	Vitest con reporte de cobertura	Al cierre de cada sprint
Cobertura de requerimientos	Requerimientos con al menos un caso de prueba automatizado entre requerimientos totales, por cien	Porcentaje	100 % de los requerimientos con regla de negocio	Matriz de trazabilidad	Al cierre de cada sprint
Densidad de defectos	Defectos encontrados entre miles de líneas de código	Defectos por millar	Menos de 5	Registro de defectos	Al cierre de cada sprint
Defectos escapados	Defectos hallados después de la entrega entre defectos totales, por cien	Porcentaje	Menos de 10 %	Registro de defectos	Al final del proyecto
Reglas garantizadas en la base de datos	Reglas con restricción o disparador entre reglas de negocio totales, por cien	Porcentaje	100 % de las reglas duras	Revisión del guion	Al cierre de cada sprint
Tiempo de respuesta	Percentil 95 del tiempo de los diez puntos de acceso más utilizados	Segundos	Menos de 3	Prueba de carga con k6	Sprint 7 y previo a la entrega
Disponibilidad	Minutos en servicio entre minutos del horario operativo, por cien	Porcentaje	Igual o mayor a 99 %	Sonda automática al punto de acceso de salud	Mensual

Nota. Las metas de tiempo de respuesta y disponibilidad provienen de los requerimientos no funcionales del proyecto. El término regla dura designa aquellas cuya violación produce datos inconsistentes.

Métricas de Proceso y de Proyecto

Tabla 7

Fichas de las métricas de proceso y de proyecto

Métrica	**Fórmula**	**Propósito**
Velocidad del equipo	Historias de usuario terminadas por sprint	Proyectar si el alcance restante cabe en el tiempo disponible
Cumplimiento de la definición de terminado	Historias que cumplen los siete criterios entre historias cerradas, por cien	Detectar avance aparente que en realidad acumula deuda técnica
Defectos por fase de detección	Conteo de defectos hallados en diseño, en pruebas y en producción	Sustentar que el costo de corrección se incrementa conforme el defecto avanza de fase
Retrabajo	Horas dedicadas a corrección entre horas totales, por cien	Cuantificar el costo de la mala calidad
Tiempo medio de corrección	Promedio de días entre el reporte de un defecto y su cierre	Evaluar la capacidad de respuesta del equipo
Avance de requerimientos	Requerimientos implementados entre requerimientos totales, por cien	Comunicar el progreso real del proyecto

Nota. Elaboración propia con base en Pressman y Maxim (2020) y Sommerville (2016).

Valores Medidos al 19 de Octubre de 2026

Tabla 8

Estado actual de las métricas

Métrica	**Valor anterior**	**Valor actual**	**Meta**	**Estado**
Pruebas automatizadas	9, todas satisfactorias	9, todas satisfactorias	Incremento por sprint	No cumplido
Casos de prueba ejecutados	26	40	Incremento por sprint	Cumplido
Cobertura de pruebas en servicios	Sin medir	Sin medir	Igual o mayor a 70 %	Pendiente
Reglas de negocio garantizadas en la base	7 de 7 reglas duras	7 de 7 reglas duras	100 %	Cumplido
Reglas verificadas con datos inválidos	9 pruebas ejecutadas	9 pruebas ejecutadas	100 % de las reglas duras	Cumplido
Restricciones de integridad por tabla	2.5	2.4	Sin meta definida; se reporta como indicador	Informativo
Avance de requerimientos	7 de 16, es decir 44 %	12 de 16, es decir 75 %	100 %	En curso
Trazabilidad de requerimientos	16 de 16 documentados	16 de 16 documentados	100 %	Cumplido
Densidad de defectos	2 defectos en 10,040 líneas, 0.20 por millar	6 defectos en 12,941 líneas, 0.46 por millar	Menos de 5 por millar	Cumplido
Tiempo medio de corrección	Menos de un día	Menos de un día en los seis defectos	Sin meta definida aún	Informativo
Tiempo de respuesta	Sin medir	Sin medir	Menos de 3 segundos	Pendiente

Nota. Las reglas duras corresponden a los requerimientos funcionales RF-04, RF-05, RF-06, RF-08, RF-09, RF-10 y RF-16. Los seis defectos registrados figuran en el plan de pruebas como DEF-01 a DEF-06. DEF-03 y DEF-04 se detectaron al conectar el sistema a una base de datos real por primera vez; DEF-05 y DEF-06, al usar los modulos de alertas y monitoreo con un rol distinto al del guion original.

El dato que exige atención no es ninguno de los que cumplen su meta, sino el que no la cumple: la cantidad de pruebas automatizadas permanece en nueve mientras el código creció un 29 %. Los cuatro módulos nuevos se verificaron de forma manual, con los casos CP-27 a CP-38, y esa verificación manual no se repite sola en cada cambio. La consecuencia práctica es que una modificación en el módulo de monitoreo puede romper la regla de los cinco minutos sin que ninguna prueba lo advierta. La acción correctiva correspondiente, automatizar los casos CP-27 a CP-40 con Vitest y Supertest, queda registrada como la primera tarea del sprint siguiente.

Los defectos DEF-05 y DEF-06, registrados el 23 de octubre de 2026, confirman ese riesgo con hechos. Ninguno de los dos lo encontró una prueba: aparecieron al usar el sistema con una cuenta distinta a la del guion con que se verificó cada módulo. DEF-05 impedía atender una alerta y, por la forma en que la transacción fallida dejaba la conexión, arrastraba a los demás módulos; DEF-06 dejaba el monitoreo inservible para todo rol que no estuviera atado a una estación. La lección que queda documentada es que verificar un módulo con un solo rol deja sin recorrer la mitad de sus caminos, y que la matriz de permisos no solo debe probarse para lo que cada rol tiene prohibido, sino también para lo que tiene permitido.

Umbrales y Acciones Correctivas

Una métrica sin una acción asociada no modifica el comportamiento del proyecto. La tabla siguiente define qué hacer cuando un valor se aparta de su meta.

Tabla 9

Umbrales de alerta y acciones correctivas

Métrica	Umbral de alerta	Acción correctiva
Cobertura de pruebas	Menor a 70 % al cierre del sprint	No se cierra el sprint hasta agregar las pruebas faltantes de los servicios modificados
Pruebas automatizadas	Un módulo nuevo sin prueba automatizada de su regla de negocio	Se automatizan sus casos antes de iniciar el módulo siguiente
Cobertura de requerimientos	Un requerimiento con regla de negocio sin caso de prueba	Se agrega el caso antes de dar la historia por terminada
Densidad de defectos	Mayor a 5 por millar de líneas	Revisión por pares obligatoria del módulo afectado en el sprint siguiente
Reglas garantizadas en la base	Menor a 100 %	La historia no cumple la definición de terminado y se reabre
Tiempo de respuesta	Mayor a 3 segundos en cualquier punto de acceso	Análisis del plan de ejecución de la consulta y revisión de índices
Disponibilidad	Menor a 99 % en el mes	Análisis de causa raíz documentado y plan de corrección
Defectos en producción	Cualquier defecto escapado	Se agrega un caso de prueba que lo reproduzca antes de corregirlo

Nota. Elaboración propia. El umbral de pruebas automatizadas se incorpora en esta edición como consecuencia directa de la medición del 19 de octubre de 2026.

Procedimiento de Recolección

Las métricas automatizables se obtienen mediante los guiones siguientes, ejecutados desde la carpeta del servidor:

```
npm test                    Ejecuta las pruebas automatizadas
npx vitest run --coverage   Genera el reporte de cobertura de lineas y
ramas
npm run db:local:reset      Recrea el esquema y verifica el guion
completo
```

Las métricas de tamaño se obtienen por conteo sobre el repositorio y por consulta al catálogo del sistema de PostgreSQL. Las métricas de proceso se registran de forma manual en la bitácora del proyecto al cierre de cada sprint. La verificación del guion contra un motor real se

realiza con PGlite, lo cual permite ejecutar las pruebas de reglas de negocio en cualquier equipo sin instalar un servidor de base de datos.

Conclusiones

El stack del SIC Transmetro se compone de siete tecnologías en la capa de presentación, once en la capa de aplicación y seis en la capa de datos, además de siete herramientas de desarrollo y aseguramiento de la calidad. Todas son de código abierto. La decisión de mayor peso sigue correspondiendo al motor de base de datos: PostgreSQL se seleccionó por una capacidad concreta, las restricciones de exclusión sobre rangos temporales, que garantizan los requerimientos RF-04 y RF-06 sin depender del código de la aplicación. La construcción de los cuatro módulos nuevos confirmó la segunda virtud de esa decisión: las reglas de los requerimientos RF-09 y RF-10 ya estaban en el motor, de modo que la interfaz de monitoreo se limitó a presentarlas.

El plan de métricas comprende siete métricas de producto, seis de proceso y de proyecto, y un conjunto de umbrales con su acción correctiva asociada. Entre las dos mediciones, el avance de requerimientos pasó del 44 % al 75 %, el código creció un 29 % y la densidad de defectos se mantuvo muy por debajo de su umbral. En contrapartida, la cantidad de pruebas automatizadas no creció, lo cual constituye la deuda técnica más relevante del proyecto en este momento y la prioridad del sprint siguiente.

Referencias

ISO/IEC. (2011). *ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.

Meta Platforms. (s.f.). *React documentation*. https://react.dev

OpenJS Foundation. (s.f.). *Node.js documentation*. https://nodejs.org/docs

PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson Education.